Esercitazione Python n. 8 -- 24 Novembre 2025

Per risolvere gli esercizi **scrivete la soluzione di ogni esercizio nel file .py relativo**, che trovate nella cartella dell'esercitazione (ad esempio, per l'esercizio 1 scrivete il vostro programma nel file Ex1.py, per l'esercizio 2, nel file Ex2.py, e così via). Notate che ogni file incorpora del codice python per eseguire alcuni test sulla funzione. NON modificate questo codice, ma **SCRIVETE SOLO il contenuto della funzione**. Per farlo usare l'ambiente Spyder. Ovviamente è possibile consultare il materiale didattico disponibile sulla pagina web del corso.

Potete riconsegnare l'esercitazione attraverso il sito classroom. L'esercitazione non verrà comunque corretta singolarmente.

In ogni esercizio, se non diversamente richiesto, potete sempre assumere che gli input forniti siano coerenti con la traccia (ad esempio, se l'esercizio chiede di dare in input alla funzione un intero positivo, potete assumere che l'input sia sempre un numero intero maggiore di zero, e non è necessario nel codice effettuare controlli per gestire casi diversi da questo).

Esercizi

- **Ex1(l)** Scrivere una funzione che prende in ingresso una lista **l** di stringhe e restituisce un insieme contenente tutti e soli i caratteri che appaiono almeno due volte in una delle stringhe. Ad esempio, se **l**=`['casa','albero','bello']`, allora l'insieme da restituire sarà `{'a','l'}`. Ovviamente, se la lista **l** in ingresso è vuota, oppure non ci sono caratteri che appaiono almeno due volte in una stringa di **l**, la funzione deve restituire l'insieme vuoto.

- **Ex2(l,c,n)** Scrivere una funzione che prende in ingresso una lista **l** di stringhe, un carattere **c** ed un numero intero **n** e restituisce una lista che contiene tutte le stringhe di **l**, nello stesso ordine in cui compaiono in **l**, escluse quelle che contengono almeno **n** volte il carattere **c**. Ad esempio, se **l** = `['palla','casse','palo']`, **c** =`'l'` ed **n** = `2` allora la funzione deve restituire la lista `['casse','palo']`. Ovviamente se la lista **l** in ingresso è vuota, la funzione deve restituire una lista vuota.

- **Ex3(l)** Scrivere una funzione che prende in ingresso una lista **l** di numeri interi positivi e restituisce una lista ottenuta modificando **l** nel seguente modo: ogni volta che un numero in **l** è più piccolo del successivo, alla lista viene aggiunta in fondo la differenza tra i due (il più grande meno il più piccolo). Ad esempio, se **l**=`[10,1,11,31,251]`, allora la lista da restituire sarà `[10,1,11,31,251,10,20,220,10,200,190]`. Si noti che il confronto fra un numero ed il successivo deve essere fatto anche per i nuovi elementi inseriti, come mostrato dall'esempio. Ovviamente, se la lista in ingresso è vuota, la funzione deve restituire una lista vuota.

- **Ex4(i)** Scrivere una funzione che prende in ingresso un insieme **i** di interi e restituisce la lista contenente i soli elementi divisibili per 5. La lista deve essere ordinata in ordine crescente. Ad esempio, se **i**=`{3,345,15,24,25,-10}`, la funzione deve restituire `[-10,15,25,345]`.
  Suggerimento: Ricordare che, dato un insieme `ins`, il seguente ciclo for
  ```
  for elem in ins:
  ```
  seleziona ad ogni iterazione un elemento diverso di `ins` (assegnandolo ad `elem`), che ogni elemento è selezionato una sola volta e che il ciclo termina quando tutti gli elementi sono stati selezionati.

- **Ex5(l)** Scrivere una funzione che prende in ingresso una lista **l** di stringhe e restituisce un insieme di tutte e sole le coppie *(x,n)* tali che *x* è una stringa di **l** ed *n* è il numero di volte che la stringa *x* appare in **l**. Notare che una coppia è semplicemente una tupla di lunghezza due. Ad esempio, se **l**=`['jkl','h','plqa','jkl','h','xkj']`, allora la funzione deve restituire l'insieme

`{('jkl',2),('h',2),('plqa',1),('xkj',1)}`. Ovviamente se la lista **l** in ingresso è vuota, la funzione deve restituire l'insieme vuoto.

- **Ex6(l)** Scrivere una funzione che prende in ingresso una lista **l** di stringhe e restituisce un insieme di coppie *(x1, x2)* tali che *x1* e *x2* sono stringhe di **l** della stessa lunghezza e *x1* è diverso da *x2*. In altri termini, l'insieme deve contenere tutte e sole le coppie formate combinando a due a due, in tutti i possibili modi, le stringhe di **l** che hanno la stessa lunghezza, escludendo i casi in cui una stringa si combina con se stessa. Ad esempio, se **l**=`['jkl','h','plqa','a','xkj']`, allora l'insieme di coppie da restituire sarà `{('jkl','xkj'),('h','a'),('a','h'),('xkj','jkl')}`. Ovviamente se la lista **l** in ingresso è vuota, la funzione deve restituire l'insieme vuoto.

- **Ex7(a, b)** Scrivere una funzione che prende in ingresso due insiemi **a** e **b** di coppie. L'insieme **a** contiene coppie di forma (nomePersona, cittàNascita), ad indicare che una persona è nata in una certa città, mentre l'insieme **b** contiene coppie di forma (città, regione), ad indicare che una città appartiene ad una certa regione. La funzione deve restituire un altro insieme contenente tutte e sole le coppie (nome, regione) che indicano in quale regione è nata una persona sulla base di quanto contenuto in **a** e **b** e se questa informazione è disponibile. Ad esempio, se **a**=`{('Giovanni', 'Napoli'),('Marco','Roma'),('Giuseppe','Rieti'),('Aldo','Torino')}` e **b**=`{('Napoli','Campania'), ('Benevento','Campania'), ('Roma',Lazio'), ('Rieti','Lazio'), ('Genova','Liguria')}`, allora l'insieme da restituire sarà `{('Giovanni','Campania'),('Marco','Lazio'),('Giuseppe','Lazio')}`. Si assuma che **a** e **b** siano insiemi che contengono solo coppie i cui elementi sono stringhe, oppure che siano vuoti. Se una città è presente in **a** ma non in **b** allora la persona nata in quella città NON deve essere nell'insieme risultato (si noti che nel risultato dell'esempio precedente non c'è una coppia relativa ad `'Aldo'`). Ovviamente, se uno dei due insiemi in ingresso è vuoto, la funzione deve restituire un insieme vuoto.

- **Ex8(l)** Scrivere una funzione che prende in ingresso una lista **l** contenente stringhe non vuote e restituisce l'insieme dei caratteri più frequenti di ciascuna stringa. Se una stringa ha più di un carattere più frequente, l'insieme deve contenere il più piccolo carattere in ordine alfabetico (Unicode). Ad esempio, se **l**=`['amaca','amaranto','rosso']` allora la funzione deve restituire `{'a','o'}`, perché `'a'` è il carattere più frequente in `'amaca'` ed in `'amaranto'` ed `'o'` è il carattere più frequente `'rosso'`. Se **l** è la lista vuota, la funzione deve restituire l'insieme vuoto `set()`.

- **Ex9(l)** Scrivere una funzione che prende in ingresso una lista **l** *non vuota* di stringhe e calcola quale carattere alfabetico minuscolo compare in più stringhe. Se ci sono più caratteri che hanno questa caratteristica, si scelga quello alfabeticamente più grande. Ad esempio, se **l** = `['casa','senape', 'ketchup','pasta']`, allora il carattere da restituire è `'s'` che compare in 3 stringhe ed è più grande di `'a'` e `'p'`, che anche compaiono in 3 stringhe. Potete assumere che gli unici caratteri alfabetici minuscoli usati nelle stringhe appartengano all'alfabeto inglese (cioè sono compresi fra `'a'` e `'z'` nell'ordinamento Unicode).
  Suggerimento: Usate un ciclo che ad ogni iterazione considera una lettera alfabetica minuscola diversa (ad es. `for lettera in 'abcdefghijklmnopqrstuvwyz':…`).

- **Ex10(l)** Scrivere una funzione che prende in ingresso una lista **l** di insiemi di numeri interi e restituisce un insieme contenente tutti e soli gli interi che appaiono in uno ed uno solo degli insiemi in **l**. Ad esempio, se x=`[{3,2,90},{2,87,23},{2,23,3}]`, allora l'insieme da restituire sarà `{90,87}` (si noti che 3, 2 e 23 compaiono in più insiemi di **l** e quindi non devono essere contenuti nel risultato). Se la lista in ingresso è vuota, la funzione deve restituire un insieme vuoto.